

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

ASSESSMENT TOOL 1 -Pseudocode Writing Task (Algorithm Design)

Course Code:	CSA03	Course Title:	Data Structures
CO Assessed:	CO5 –Naturalization (BL4, BL6)	Assessment:	Assessment Tool 1- Pseudocode Writing Task (Algorithm Design)
Weightage:	30% of CIA	Total Marks:	50
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem. (Bloom Level -BL4, BL6)		
Student Name:	T.VAISHNAVI	Reg. No.:	192421370
Section / Batch:	B-TECH(IT)	Date:	21/08/2026

Code of Conduct

I **T.VAISHNAVI(192421370)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 5 Coding challenge Total: 50 Marks.
- Evaluation of Answer Quality -Checking whether the answer includes: Problem Understanding, Algorithm Design, Use of Control Structures, Pseudocode Structure and Completeness
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Graph Terminologies - Topological Sort - MST	MST	1,2	20
Shortest path Algorithm- Floyd- Warshall	Floyd and warshal Algorithms	3,4	20
GPS & AI Pathfinding-Graph Traversal	BFS, DFS	5	10
GRAND TOTAL:		50Marks	

Annexure A – Pseudocode Writing Task (Algorithm Design)

Date: 21/04/2026 COS - AT 1

Name: T. Vishnu
Reg No: 192421870
Course Code: CSA0312
Course Name: Data Structure

① Topological Sort - Higher Priority

Pseudocode :-
TOPOLOGICAL-SORT(G)

Calculate in-degree of all Vertices
Insert all in-degree 0 Vertices into Priority Queue

while PQ not empty

u ← Remove highest-priority Vertex
 Print u
 for each neighbor v of u
 in-degree[v]--
 if in-degree[v] = 0
 Insert v into PQ

if all Vertices not Printed
 Print "cycle exists"

END

② Kruskal's Algorithm

Pseudocode
KRUSKAL(G)

Sort edges by increasing weight

MST ← empty

for each vertex v
 MAKE-SET(v)

for each edge (u, v) in sorted order

 if FIND(u) ≠ FIND(v)

 Add (u, v) to MST
 UNION(u, v)

if MST has $V-1$ edges

break
Print MST and total cost

END

③ Modified Dijkstra - Avoid Nodes

Pseudocode :-

DIJKSTRA(G , Source, target, Avoid)

Set distance[Source] = 0

Set all other distances = ∞

while Priority Queue not empty

$u \leftarrow$ minimum distance node

 if u in Avoid

 continue

 if $u = \text{target}$

 break

 for each neighbour v of u

 if v not in Avoid

 Update distance [v]

if target unreachable

 Print "No Valid Path"

else

 Print shortest Path

END

④ Space-optimized Floyd - Warshall

Pseudocode

FLOYD-WARSHALL(G)

Initialize distance matrix D

for $k = 1$ to V

for $i = 1$ to V

for $j = 1$ to V

$D[i][j] \leftarrow \min(D[i][j],$

$D[i]$

$[k] + D[k][j])$

Print D

END

⑤ BFS with Early Stopping

Pseudocode:-

BFS-EARLY-STOP($G, source, target$)

Create queue Q

visited $\leftarrow$ empty set

parent $\leftarrow$ empty map

level $\leftarrow$ empty map

Add source to visited level[source] $\leftarrow 0$

Enqueue source into Q

while Q is not empty do

$u \leftarrow$ Dequeue(Q)

if $u = target$ then break

for each neighbour V of u do

if V is not in visited then

Add V to visited

Parent[v] ← u

level[v] ← level[u] + 1

if v = target then

Reconstruct Path using Parent

Print "Target found"

Print Path

Print "connection level="; level[v] return

Enqueue v into Q

Print "Target not found"

END

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____